

1. Aw Shii, Here we flow again...

Seeing Motion Like Humans Do

Think of merging onto a highway at dusk: your brain instinctively tracks the speed of approaching cars using subtle visual cues, not raw speedometers. Optical flow replicates this biological motion sense for machines, solving critical problems other sensors miss.

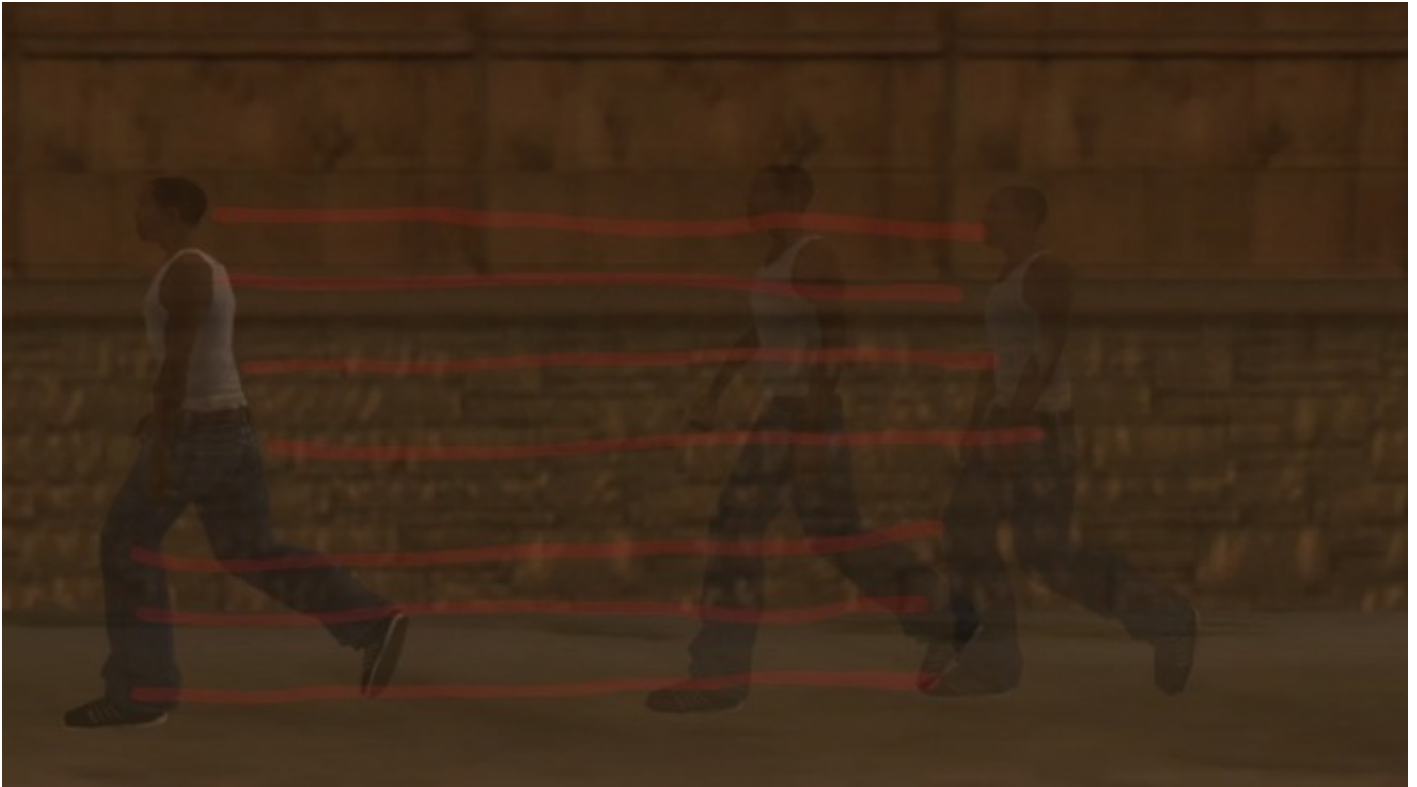
- Sudden Obstacles: Like spotting a skateboarder cutting across traffic (as in Chalmers University's collision detection system), optical flow detects approaching objects **30% faster than frame-by-frame detection**.
- Parking Lot Navigation: Ever parallel park by gauging car bumpers' movement in your mirrors? Flow vectors create real-time "distance maps" between vehicles.
- Rainy Night Driving: New methods like **SemARFlow** maintain 85% accuracy in downpours where LiDAR fails—critical when windshield wipers struggle.
- Key Insight: Humans use motion cues first, details later—optical flow brings this biological advantage to machine.



SUBTASK 1: IMPLEMENTING OPTICAL FLOW

CJ NEEDS HELP!

You are expected to implement the Lucas-Kanade sparse optical flow estimation algorithm specified in this [article](#) on clips of your choice from this [video](#). You are allowed to use basic OpenCV functions: `cv2.imread`, `cv2.imshow`, etc., and you may additionally use `cv2.goodFeaturesToTrack`. You may refer OpenCV documentation.



BONUS: Implement dense Optical Flow estimation algorithm on the video.

SUBTASK 2: GOING WITH THE FLOW

Let us now dive into a very interesting application of optical flow methods: moving your ego (a robot or an autonomous car) towards a target while avoiding obstacles. Your task here is to use the Lucas-Kanade algorithm in conjunction with a few methods in order to implement a make-shift controller that reaches a given goal in a [cool simulator](#).

Before proceeding with the task, you are required to pull and set up the docker container as mentioned in the compulsory task. Operate only from the home directory.

Launch the container and do a [pip install of pybullet](#), then create a python file where you will be coding up most or all of the following task.

Objective: Use sparse optical flow (Lucas-Kanade method) to navigate to the end of the racetrack avoiding the obstacles in the provided setup ([using the given script](#)).

You must follow these steps (refer to the paper linked below for more clarity):

- Set up a camera feed and capture RGB frames from Pybullet's agent camera.
- Compute optical flow between consecutive frames to identify motion vectors.
- Detect Focus of Expansion (FOE) to infer agent motion direction and distinguish ego-motion from obstacle motion.
- Generate a visual potential field where obstacles create repulsive vectors and the target (e.g., a doorway) creates an attractive vector.
- Integrate with Pybullet's API to steer the agent using gradient based control.

You must go through this [this paper](#) and understand its math (at least at a high-level) to accomplish this. Apart from your implementation of Lucas-Kanade method, you may use any library function from OpenCV and NumPy. Go through their documentation.

Use as many resources as you want (*including LLMs*) for this subtask.



Figure 1: Camera feed

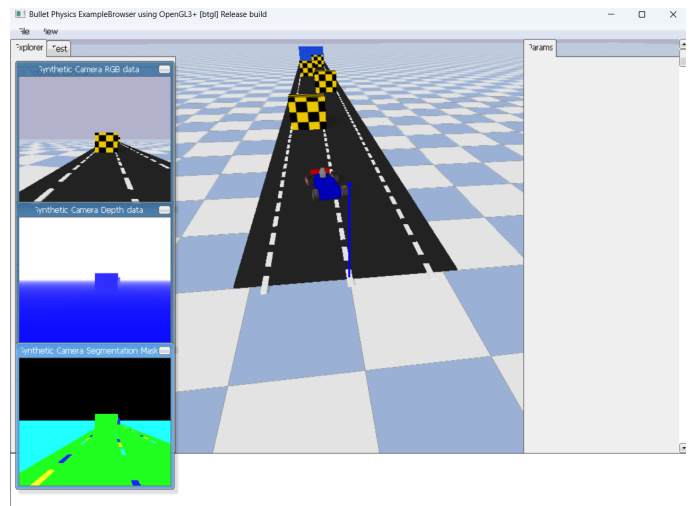


Figure 2: Setup